

Linear vs. Logistic Regression and Threshold Tuning

Lab Report

Student Name	Akshay Kumar M
Student ID (SRN)	PES1UG24CS045
Course	UE24CS352A: Machine Learning
Lab	Lab 5: Linear vs. Logistic Regression and Threshold Tuning
Submission Date	September 03, 2026

1. Introduction

This lab explores the difference between linear regression and logistic regression when applied to binary classification problems. A small synthetic dataset with an injected extreme outlier is used first to visually demonstrate why linear regression is a poor fit for classification. The lab then builds a complete, reusable scikit-learn Pipeline (StandardScaler, SelectKBest, and LogisticRegression) on the real-world Wine Quality (Red) dataset, and studies how shifting the classification decision threshold away from the default of 0.5 trades off Precision against Recall.

2. Methodology

Part 1 - Toy Dataset: A 1D dataset of 20 points ('Hours Studied') was generated with NumPy's random uniform distribution (seed = 42) and labelled Pass (1) if Hours > 5, else Fail (0). A single extreme outlier at X = 50 (label 1) was appended to simulate a data-entry anomaly. A LinearRegression model was fit on this data and its predictions plotted against a fixed 0.5 decision line.

Part 2 - Logistic Comparison: A LogisticRegression model (random_state = 42) was fit on the identical toy dataset. Probabilities were generated over a dense range of X values (0 to 50) using predict_proba() to trace the sigmoid curve.

Part 3 - Full Pipeline: The UCI Wine Quality (Red) dataset (1,599 samples, 11 physicochemical features) was loaded and converted into a binary target: 'good_quality' = 1 if quality > 5, else 0. The data was split 70/30 (train/test) with stratification on the target (random_state = 42). A scikit-learn Pipeline chained StandardScaler (feature scaling), SelectKBest with the ANOVA F-test (f_classif, k = 8) for feature selection, and LogisticRegression (random_state = 42) as the classifier. predict_proba() was used to extract raw positive-class probabilities on the test set, and three manual thresholds (0.3, 0.5, 0.7) were applied to generate binary predictions, each evaluated with Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix.

3. Results and Analysis

3.1 Visual Comparison: Linear vs. Logistic Regression

On the toy dataset (shape (21, 1)), the fitted LinearRegression model produced predictions ranging from 0.323 to 1.493 - the maximum value clearly exceeds the valid probability range of [0, 1]. The outlier at X = 50 pulls the regression line's slope so strongly that most of the ordinary data points (X between 0 and 10) are compressed close together near the bottom of the line and sit on the wrong side of, or dangerously close to, the 0.5 decision boundary - several genuine 'Pass' points near X = 5-6 are predicted well below 0.5.

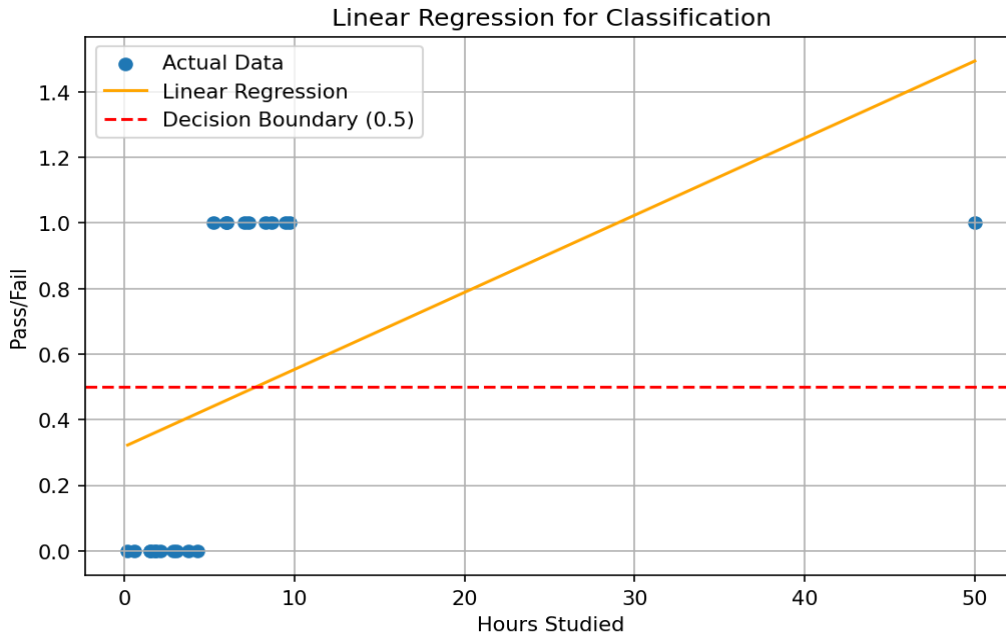


Figure 1. Linear Regression fit on the toy dataset with an outlier at X = 50.

In contrast, the LogisticRegression model's sigmoid curve stays bounded within $[0, 1]$ regardless of the outlier's magnitude. The S-curve keeps its transition centred close to $X = 5$ (the true separating point in the data), so the decision boundary is not distorted the way the linear model's was. This is the core reason logistic regression is preferred for classification: it models the log-odds of the outcome rather than the outcome directly, which naturally squashes predictions into a valid probability range.

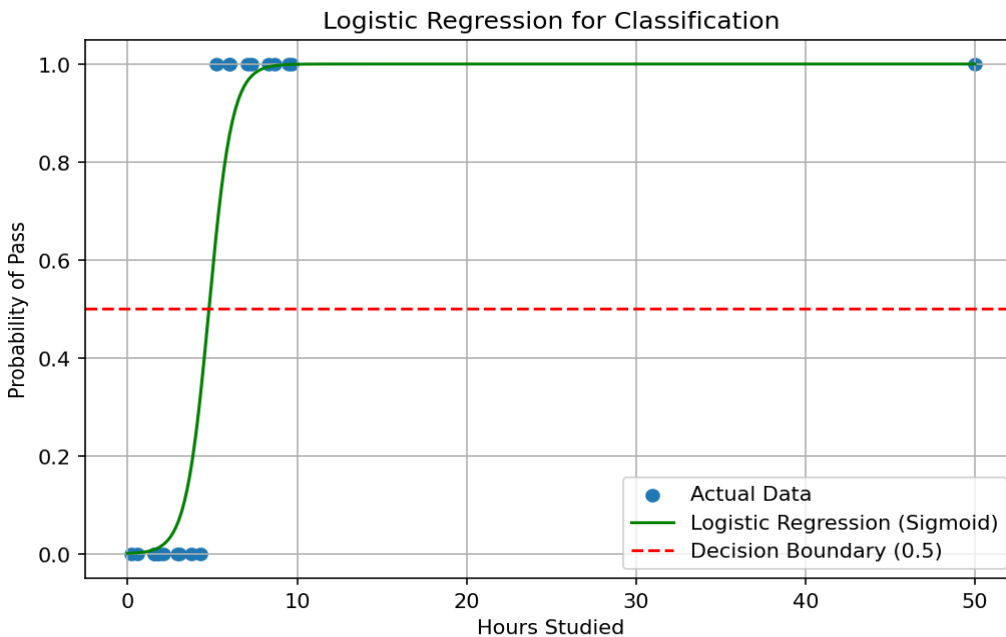


Figure 2. Logistic Regression sigmoid curve on the same toy dataset.

3.2 Performance Table (Wine Quality Pipeline)

The pipeline was trained on (1119, 11) training samples and evaluated on (480, 11) test samples.

Threshold	Accuracy	Precision	Recall	F1-Score
0.3	0.700	0.657	0.922	0.767
0.5	0.731	0.752	0.743	0.748
0.7	0.694	0.853	0.518	0.644

3.3 Threshold Trade-offs

As the threshold increases from 0.3 to 0.7, Recall falls steadily ($0.922 \rightarrow 0.743 \rightarrow 0.518$) while Precision rises steadily ($0.657 \rightarrow 0.752 \rightarrow 0.853$). This is the expected inverse relationship: a lower threshold accepts more borderline probabilities as 'good quality', catching more true positives (higher Recall) but also letting in more false positives (lower Precision). A higher threshold is stricter, so only high-confidence predictions are labelled positive, which raises Precision at the cost of missing more true positives. F1-Score, which balances both, peaks at the default 0.5 threshold (0.748) in this run, making 0.5 a reasonable general-purpose choice, though the best threshold ultimately depends on which type of error is costlier in context (see Question 2 below).

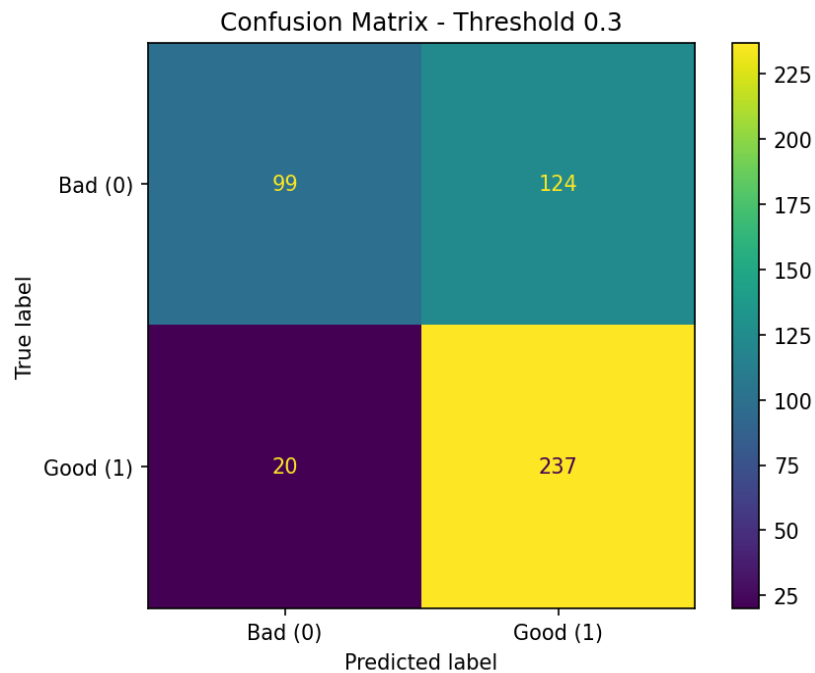


Figure 3. Confusion Matrix - Threshold 0.3.

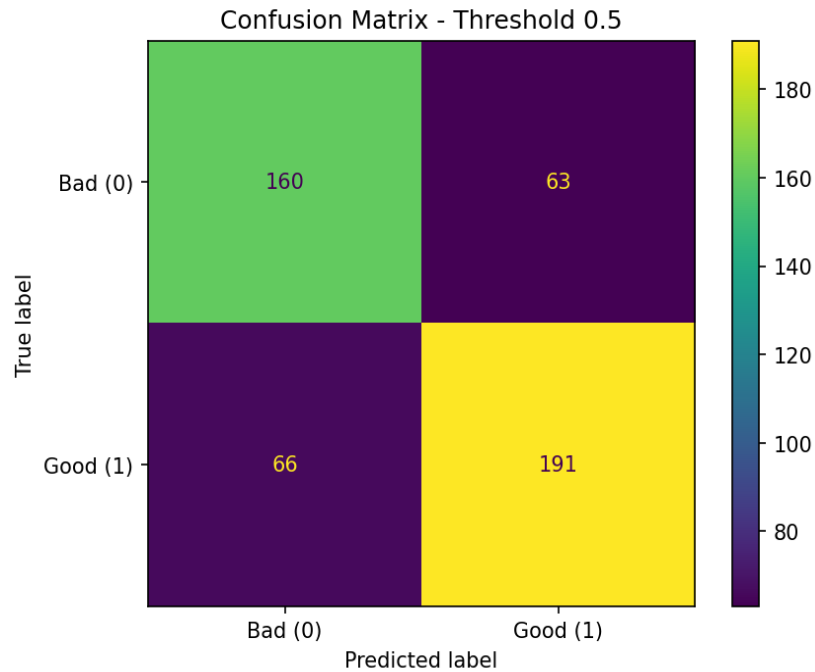


Figure 4. Confusion Matrix - Threshold 0.5.

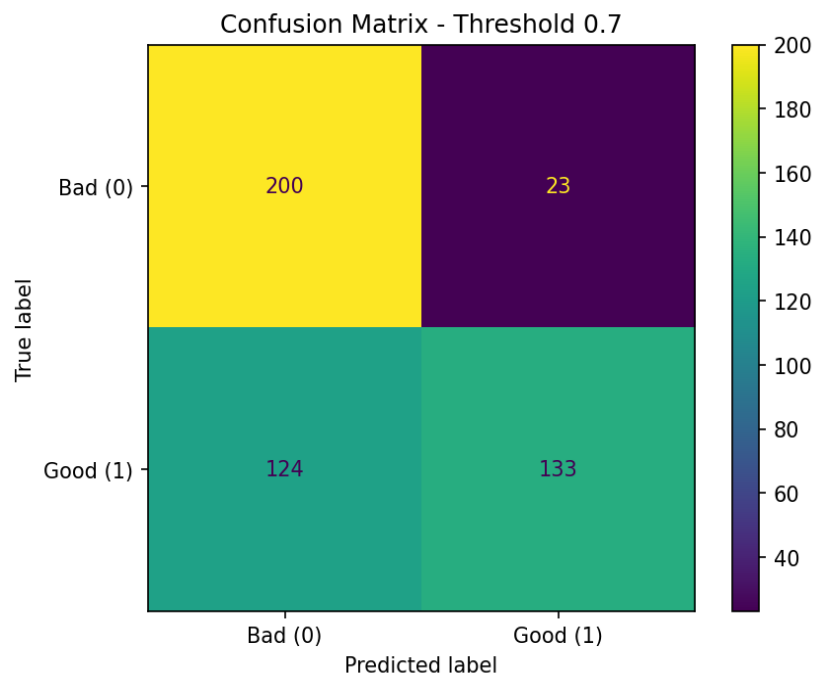


Figure 5. Confusion Matrix - Threshold 0.7.

4. Conclusion

This lab confirmed that linear regression is unsuitable for classification because its unbounded output range makes it highly sensitive to outliers and unable to represent valid probabilities, whereas logistic regression's sigmoid function bounds predictions to $[0, 1]$ and resists distortion from extreme values. On

the real-world Wine Quality dataset, a complete preprocessing-to-classification pipeline achieved reasonable performance, and adjusting the decision threshold demonstrated a clear, controllable trade-off between Precision and Recall. Threshold selection should therefore be guided by the relative business cost of false positives versus false negatives rather than defaulting to 0.5 blindly.

5. Answers to Questions

Q1. Precision vs. Recall trade-off

Precision = $TP / (TP + FP)$ and Recall = $TP / (TP + FN)$. Raising the threshold makes the classifier more conservative about predicting the positive class - only samples with high predicted probability are labelled positive. This shrinks the pool of predicted positives, so a larger fraction of them tend to be correct (Precision rises), but many true positives with moderate probability now fall below the threshold and are missed (Recall falls). Lowering the threshold does the opposite: more samples are labelled positive, catching more true positives (Recall rises) but also admitting more false positives (Precision falls). Because both metrics are computed from the same underlying probability ranking, moving the cut point can only redistribute errors between FP and FN, not eliminate both simultaneously - hence the inverse relationship observed in Section 3.3.

Q2. Contextual Threshold Selection

In the Wine Quality context, a **False Positive** means the model labels a wine 'good quality' when it is actually not, and a **False Negative** means the model labels a genuinely good wine as 'not good'. Consider a winery using this model to pre-screen batches before an expensive manual sommelier review: missing a truly good batch (False Negative) means a marketable product is discarded or demoted, a real revenue loss. Wrongly flagging a mediocre batch as 'good' (False Positive) is comparatively cheap, since it just gets caught later at manual review. In this scenario, a lower threshold (e.g., 0.3) is preferred because it maximizes Recall, ensuring almost all good-quality batches are forwarded for review even if it means passing along some average ones too.

Q3. The Problem with MSE for a Sigmoid Model

Mean Squared Error is defined as the average of $(y - \hat{y})^2$, where $\hat{y} = \text{sigmoid}(z)$. Because the sigmoid function is non-linear, substituting it into the MSE formula produces a cost function that is **non-convex** with respect to the model's weights - its surface has multiple local minima and flat, 'saturated' regions where the sigmoid's gradient is close to zero (at very large positive or negative z). Gradient descent run on this bumpy surface can easily get stuck in a local minimum or stall in a flat plateau, making the optimization slow, unstable, and dependent on initialization, rather than reliably reaching the global optimum.

Q4. Binary Cross-Entropy (Log-Loss)

Log-Loss is defined as $-[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$. Its logarithmic shape means the penalty grows very slowly when a prediction is close to the correct label, but grows sharply, approaching infinity, as a confidently wrong prediction approaches the opposite extreme (e.g., predicting $\hat{y}$ near 0 when the true label $y = 1$). A marginally incorrect prediction (say $\hat{y} = 0.45$ when $y = 1$) is penalized mildly, whereas a confidently incorrect prediction ($\hat{y} = 0.01$ when $y = 1$) is penalized severely. This asymmetric, steep penalty for confident mistakes pushes the model to be well-calibrated rather than simply 'often right'.

Q5. Gradient Descent Mechanics and Convexity

Unlike MSE combined with a sigmoid, the Log-Loss cost function is **convex** with respect to the logistic regression weights - it has a single global minimum and no local minima or saddle points to get trapped in.

This convexity guarantees that, at every point on the cost surface, the gradient points in a direction that consistently reduces the cost, so gradient descent can take a direct, well-behaved path downhill. As a result, for a suitable learning rate, gradient descent on Log-Loss converges reliably and efficiently to the optimal weights, which is precisely why Log-Loss (not MSE) is the standard cost function paired with logistic regression.